microPAM_V3 setup guide for Windows and macOS

This guide describes how the microPAM_V3 user package is installed on a Windows or macOS computer, how the workbench is started, and how firmware is uploaded to the recorder. The workbench is a Python program that uploads firmware, edits the recorder configuration, talks to the recorder over USB, and reads, plots and converts the recordings. README.md (or README.pdf) describes the recorder, its configuration and its file format in full.

The setup installs nothing system-wide apart from Python itself. All Python packages go into a private environment, the folder `Python/.venv` inside the package (about 150 MB), and deleting the package folder removes them again.

1. Overview

Step	Windows	macOS
Install Python 3.10 or later with tkinter	python.org installer	python.org installer, or Homebrew `python` and `python-tk`
Unzip the package	to a short path, e.g. `C:\microPAM_V3`	anywhere, e.g. `~/microPAM_V3`
Create the Python environment	double-click `setup.cmd`	`bash scripts/setup.sh` in Terminal
Start the workbench	double-click `workbench.cmd`	`Python/.venv/bin/python Python/workbench.py`
Upload ready-built firmware (`.uf2`)	yes	yes
Build firmware from source (optional)	with Arduino IDE 2	not supported by the workbench

An internet connection is needed once, while the setup downloads numpy, matplotlib and pyserial. No USB driver is needed on either system, as the recorder appears as a standard USB serial device and, in its boot loader, as a USB drive.

2. Windows

The package is tested on Windows 10 and 11.

2.1 Install Python

1. Download the current Python 3 installer for Windows from https://www.python.org/downloads/.
2. Run it with the default options. These include tkinter (the window toolkit of the workbench) and the `py` launcher, through which the setup finds Python. The option "Add python.exe to PATH" is not required.

The `python` command that Windows offers before any Python is installed belongs to the Microsoft Store and only opens the Store. The setup recognizes it and passes it over, so that a Python from the Store is not needed.

2.2 Unzip the package

Unzip `microPAM_V3_<version>.zip` to a folder with a short path, for example `C:\microPAM_V3` or `Documents\microPAM_V3`. Some files inside the Python packages have long names, and under a folder path of about 200 characters their full names pass the 260-character limit of Windows, whereupon matplotlib cannot be imported. The folder must also

keep its name `microPAM_V3`, because Arduino requires the sketch folder to carry the name of `microPAM_V3.ino`; this matters only for building from source.

2.3 Create the Python environment

Double-click `setup.cmd` in the package folder. A console window opens, the setup creates `Python\.venv`, installs the packages listed in `Python\requirements.txt`, checks that they can be imported, and reports whether Arduino was found. The window stays open until a key is pressed. A successful run ends with

```
Python\.venv ready.
Arduino not found: firmware can be uploaded from .uf2 files, but not built
from source. ...
```

where the Arduino line is only a note: Arduino is not needed for using the recorder.

If the pinned package versions have no installation files for the installed Python version, the current versions of numpy, matplotlib and pyserial are installed instead. If the installation fails, the incomplete environment is removed, so that the next run starts afresh. To recreate the environment deliberately, for example after a Python upgrade, one runs from a command prompt in the package folder

```
setup.cmd -Force
```

2.4 Start the workbench

Double-click `workbench.cmd`. If step 2.3 was skipped, the setup runs first, by itself. The workbench opens with the Firmware tab in user mode; developer mode, which needs Arduino (Section 5), is selected on that tab. From a PowerShell prompt, the workbench is started by

```
scripts\workbench.ps1 [-Dev]
```

3. macOS

The setup script is written for the bash and the tools that come with macOS. It has been checked for syntax but not yet run on a Mac, and reports of problems are welcome.

3.1 Install Python

Either of the following provides Python with a working tkinter:

- the macOS installer from https://www.python.org/downloads/, with default options (recommended, as tkinter is included);
- Homebrew, with both formulas:

  ```
  brew install python python-tk
  ```

  Homebrew's `python` alone has no tkinter, and the setup then reports that no suitable Python was found.

The `/usr/bin/python3` that comes with the Xcode command line tools is Python 3.9, which is too old; the setup passes it over. The setup also requires Tk 8.6 or later, because the Tk 8.5 that older macOS versions carry does not run the workbench reliably.

3.2 Unzip the package

Double-click the zip file in Finder, and move the folder `microPAM_V3` to a place of one's choice, for example the home folder. There is no path length limit on macOS.

3.3 Create the Python environment

1. Open Terminal (Applications > Utilities > Terminal).
2. Change to the package folder: type `cd` followed by a space, drag the `microPAM_V3` folder from Finder onto the Terminal window, and press Return.
3. Run

   ```
   bash scripts/setup.sh
   ```

The script searches for `python3.14` down to `python3.10` on the `PATH`, then `python3`, then python.org installations in `/Library/Frameworks/Python.framework`, and uses the first one that is 3.10 or later and has tkinter on Tk 8.6 or later. It then creates `Python/.venv`, installs the packages, checks the imports and reports whether Arduino was found, as on Windows. To recreate the environment one runs

```
bash scripts/setup.sh --force
```

The script must be started as `bash scripts/setup.sh` and not as `./scripts/setup.sh`. The zip is made on Windows and does not carry the Unix permission that marks a file as executable, so that the short form fails with "permission denied". Started through `bash`, the script also needs no clearance by Gatekeeper, the macOS check of downloaded programs.

3.4 Start the workbench

In Terminal, in the package folder:

```
Python/.venv/bin/python Python/workbench.py
```

The workbench opens with the Firmware tab in user mode. Developer mode stays disabled on macOS, because it builds and uploads through the PowerShell scripts of the package.

4. Connecting the recorder and uploading firmware

4.1 Selecting the serial port

Connect the recorder by USB. In the workbench, the serial port is selected in the bar at the top and opened with Connect.

System	Port name of the recorder
Windows	`COMn`, e.g. `COM41`, described as "USB Serial Device"
macOS	`/dev/cu.usbmodemNNNN`

On macOS each USB serial device appears twice, as `/dev/cu.*` and as `/dev/tty.*`; the `cu` device is the one to choose. Ports such as `/dev/cu.Bluetooth-Incoming-Port` belong to the computer and not to the recorder. If it is unclear which port is the recorder, one may compare the list before and after plugging it in. The workbench remembers the board by its USB serial number and finds it again after a reboot, even when the port name has changed.

Only one program can hold the serial port at a time. A serial monitor in the Arduino IDE or in another terminal program must be closed before the workbench can connect.

4.2 Uploading a ready-built firmware file

The folder `firmware/` of the package holds ready-built files for both boards and both recording modes:

File	Board and mode
`microPAM_V3_<version>_rp2040_bin.uf2`	RP2040 Adalogger, compressed recording (`.bin`)
`microPAM_V3_<version>_rp2040_wav.uf2`	RP2040 Adalogger, raw recording (`.wav`)
`microPAM_V3_<version>_rp2350_bin.uf2`	RP2350 HSTX, compressed recording (`.bin`)
`microPAM_V3_<version>_rp2350_wav.uf2`	RP2350 HSTX, raw recording (`.wav`)

Compressed recording stores the same samples losslessly in about half the space, and the workbench (Data tab, Convert to WAV) converts `.bin` files to WAV.

1. Select the serial port of the recorder (Section 4.1).
2. On the Firmware tab (user mode), choose the `.uf2` file and press Upload.

The workbench checks that the file consists of valid UF2 blocks and names the chip it was built for. It then releases the serial port and resets the board into its boot loader through that port. The board reappears as a USB drive (`RPI-RP2` for the RP2040, `RP2350` for the RP2350), the file is copied onto it, and the board restarts with the new firmware. Only a drive that carries the boot loader's `INFO_UF2.TXT` is written to, and a file built for the other chip is refused before anything is written. After the upload the workbench reconnects within 10 s and switches to the Device tab, where the boot messages appear.

A board that does not answer on its serial port, for example because its firmware has crashed, is put into the boot loader by hand: disconnect the battery, connect USB, hold BOOT (the button furthest from the USB connector), press RESET briefly and release BOOT. The upload then copies the file without the serial reset. Alternatively, one copies the `.uf2` file onto the drive that appears, in Explorer or Finder. On macOS, Finder may report that the drive was not ejected properly when the board restarts; this is expected and harmless.

4.3 Configuration and data

The Configuration tab edits the recording parameters, either in a local `config.txt` file or on the board over the serial port. The Data tab browses recordings on a local folder or on the microSD card in a card reader, which appears as a drive letter on Windows and under `/Volumes` on macOS. These functions work in the same way on both systems and are described in README.md.

5. Building firmware from source (optional, Windows only)

Building from source is only needed for settings that are compiled into the firmware (in `config.h`); the ready-built files cover both recording modes with the defaults. Without Arduino, the developer mode of the Firmware tab is disabled. To enable it:

1. Install Arduino IDE 2 from https://www.arduino.cc/en/software.
2. In File > Preferences, add this URL to "Additional boards manager URLs": `https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json`
3. In Tools > Board > Boards Manager, install "Raspberry Pi Pico/RP2040/RP2350" by Earle F. Philhower (tested with 5.6.0).
4. In the workbench, press "Check again" on the Firmware tab, or restart it.

A stand-alone arduino-cli on the `PATH`, or one named by the environment variable `ARDUINO_CLI`, is used as well. The setup reports which arduino-cli it found and whether the board package is installed.

On macOS, `setup.sh` reports an Arduino IDE 2 in `/Applications` or `~/Applications` and the board package in `~/Library/Arduino15`, but the workbench cannot build with it. One may instead open `microPAM_V3.ino` in the Arduino IDE, select the board (Adafruit Feather RP2040 Adalogger, or Adafruit Feather RP2350 HSTX with 8 MB PSRAM) and compile and upload there.

6. Updating and removing

- **New package version**: unzip it into a new folder and run the setup there. The old folder, with its environment, can then be deleted. Configuration files and recordings are not stored in the package folder unless one has put them there.
- **New Python version**: run the setup with `-Force` (Windows) or `--force` (macOS), which rebuilds `Python/.venv` with the newest suitable Python found.
- **Removal**: delete the package folder. Python itself is removed in the usual way of the system (Windows: Settings > Apps; macOS: the python.org uninstall notes, or `brew uninstall python-tk python`).

7. Troubleshooting

Symptom	Cause and remedy
"No Python 3.10+ with tkinter found"	Python is missing, too old, or lacks tkinter. Install it as in Section 2.1 or 3.1 and run the setup again; on macOS with Homebrew, install `python-tk` as well.
"pip install failed (no internet connection?)"	The packages could not be downloaded. Check the connection, or a proxy, and run the setup again.
Windows: "The new environment cannot import its packages", folder path is long	The path length limit of Windows (Section 2.2). Move the folder to a short path such as `C:\microPAM_V3` and run the setup again.
macOS: "permission denied" when starting the setup	It was started as `./scripts/setup.sh`. Use `bash scripts/setup.sh`.
macOS: "No such file or directory" when starting the workbench	Terminal is not in the package folder, or the setup has not run. Change to the folder (Section 3.3, step 2) and run the setup.
The recorder does not appear among the serial ports	Check the USB cable (some cables carry power only), press RESET, and refresh the port list with the ↺ button beside it. A board in its boot loader shows a USB drive instead of a serial port.
"cannot open COMn" or "Resource busy"	Another program holds the port. Close any other serial monitor, or disconnect in the workbench before using another program.
The upload waits for the boot drive and then fails	The board did not enter the boot loader. Put it there by hand (BOOT and RESET, Section 4.2) and upload again.
"the file is built for RP2350, but the board is an RP2040" (or the reverse)	The file was built for the other board. Choose the file with the matching `rp2040` or `rp2350` in its name.

8. Command line reference

The workbench functions for upload and conversion are also available without the window.

Task	Windows	macOS
Set up	`setup.cmd [-Force]`	`bash scripts/setup.sh [--force]`
Start the workbench	`workbench.cmd`	`Python/.venv/bin/python Python/workbench.py`
Upload a `.uf2` file	`Python\.venv\Scripts\python.exe Python\uf2_upload.py FILE.uf2 [--port COMn]`	`Python/.venv/bin/python Python/uf2_upload.py FILE.uf2 [--port /dev/cu.usbmodemNNNN]`
Convert `.bin` to WAV	`Python\.venv\Scripts\python.exe Python\bin2wav.py DEST FILE_OR_FOLDER ...`	`Python/.venv/bin/python Python/bin2wav.py DEST FILE_OR_FOLDER ...`